

Universidad Francisco de Vitoria

Criptografía

Criptografía Poscuántica

Implementación de CRYSTALS-Dilithium

Firmas digitales resistentes a ataques cuánticos

David Sanz Fuertes

María Rivas Ramos

Javier Fernández Meroño

Fecha de entrega: 8 de mayo de 2026

Índice general

1. Introducción	2
2. Fundamentos Matemáticos	3
2.1. Retículas	3
2.2. El anillo de polinomios R_q	3
2.3. Module-SIS (M-SIS)	4
2.4. Transformada Número-Teórica (NTT)	4
2.5. Fiat-Shamir con abortos	5
3. El Esquema CRYSTALS-Dilithium	6
3.1. Descripción general	6
3.2. Parámetros	6
3.3. Generación de claves: KeyGen	6
3.4. Firma: Sign	7
3.5. Verificación: Verify	7
3.6. Repo Github	7
4. Conclusiones	8

Capítulo 1

Introducción

La llegada de la computación cuántica supone una amenaza directa para los esquemas de firma digital actuales. Algoritmos como Diffie-Hellman o RSA-PSS basan su seguridad en problemas matemáticos, la factorización de enteros y el logaritmo discreto, que un ordenador cuántico podría resolver eficientemente mediante el algoritmo de Shor [1]. Para anticiparse a esta amenaza, el NIST inició en 2016 un proceso de estandarización de criptografía poscuántica [4], del que CRYSTALS-DILITHIUM salió seleccionado como esquema de firma digital principal, actualmente recogido en el estándar FIPS 204 [6]. En este trabajo se presentan los fundamentos matemáticos que sustentan Dilithium, las retículas modulares, el problema M-SIS y la técnica de *Fiat-Shamir con abortos*, junto a una implementación funcional en Python.

Capítulo 2

Fundamentos Matemáticos

2.1. Retículas

Definición 2.1 (Retícula). Sea $\mathbf{B} = \{\mathbf{b}_1, \dots, \mathbf{b}_n\} \subset \mathbb{R}^m$ un conjunto de vectores linealmente independientes. La retícula generada por $\mathbf{B}$ es

$$\mathcal{L}(\mathbf{B}) = \left\{ \sum_{i=1}^n z_i \mathbf{b}_i \mid z_i \in \mathbb{Z} \right\}.$$

La seguridad de los esquemas de firma en retículas descansa sobre el problema SIS (*Short Integer Solution*):

Problema 2.1 ($\text{SIS}_{n,m,q,\beta}$). Dada una matriz $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, encontrar un vector no nulo $\mathbf{z} \in \mathbb{Z}^m$ tal que

$$\mathbf{A}\mathbf{z} = \mathbf{0} \pmod{q} \quad \text{y} \quad \|\mathbf{z}\|_\infty \leq \beta.$$

Nota: $\xleftarrow{\$}$ significa muestreo aleatorio.

Observación 2.1. Resolver SIS es tan difícil como resolver instancias del problema SVP (*Shortest Vector Problem*) en retículas, considerado intratable incluso para ordenadores cuánticos [3].

2.2. El anillo de polinomios R_q

Dilithium opera sobre el anillo

$$R_q = \mathbb{Z}_q[x] / (x^n + 1),$$

con $n = 256$ y $q = 8\,380\,417$ (primo tal que $q \equiv 1 \pmod{2n}$). Los elementos de R_q son polinomios de grado menor que n con coeficientes en $\{0, 1, \dots, q-1\}$.

2.3. Module-SIS (M-SIS)

Problema 2.2 (M-SIS $_{n,k,l,q,\beta}$). *Dada una matriz $\mathbf{A} \in R_q^{k \times l}$, encontrar un vector no nulo $\mathbf{z} \in R_q^l$ tal que*

$$\mathbf{A}\mathbf{z} = \mathbf{0} \quad y \quad \|\mathbf{z}\|_\infty \leq \beta.$$

- $\mathbf{A} \in R_q^{k \times l}$: matriz pública aleatoria con entradas en R_q .
- $\mathbf{z} \in R_q^l$: vector solución formado por l polinomios de R_q .
- $\mathbf{A}\mathbf{z} = \mathbf{0}$: producto de polinomios en el anillo, reduciendo módulo q y módulo $x^n + 1$.
- $\|\mathbf{z}\|_\infty \leq \beta$: todos los coeficientes de $\mathbf{z}$ deben ser pequeños, acotados por β .

Los parámetros k y l controlan el nivel de seguridad. Dilithium emplea también M-LWE para proteger la clave privada.

2.4. Transformada Número-Teórica (NTT)

La NTT (*Number Theoretic Transform*) es una versión de la FFT adaptada para trabajar con enteros módulo q en lugar de números complejos. Se utiliza para multiplicar polinomios de forma eficiente dentro del anillo R_q .

El problema de multiplicar dos polinomios directamente es que cuesta $\mathcal{O}(n^2)$ operaciones, ya que cada coeficiente del resultado depende de todos los coeficientes de ambos polinomios. La NTT resuelve esto en tres pasos:

1. Transformar ambos polinomios al dominio NTT, con coste $\mathcal{O}(n \log n)$.
2. Multiplicar los polinomios transformados coeficiente a coeficiente, con coste $\mathcal{O}(n)$.
3. Aplicar la transformada inversa (INTT) para recuperar el resultado, con coste $\mathcal{O}(n \log n)$.

El coste total es $\mathcal{O}(n \log n)$, frente a $\mathcal{O}(n^2)$ de la multiplicación directa. Con $n = 256$ esto supone una mejora significativa en rendimiento.

La fórmula de la transformada evalúa el polinomio $f = f_0 + f_1x + \dots + f_{n-1}x^{n-1}$ en n puntos concretos de $\mathbb{Z}_q$:

$$\hat{f}_j = \sum_{i=0}^{n-1} f_i \cdot \zeta^{(2j+1)i} \pmod{q}, \quad j = 0, \dots, n-1,$$

donde ζ es una raíz primitiva $2n$ -ésima de la unidad en $\mathbb{Z}_q$, es decir, un entero tal que $\zeta^{2n} \equiv 1 \pmod{q}$ y ninguna potencia menor cumple esa condición. Hace el papel de $e^{2\pi i/n}$ en la FFT clásica, pero operando íntegramente en $\mathbb{Z}_q$. Su existencia está garantizada porque $q \equiv 1 \pmod{2n}$, condición que cumple $q = 8\,380\,417$ con $n = 256$.

2.5. Fiat-Shamir con abortos

La técnica *Fiat-Shamir con abortos*, introducida por Lyubashevsky [2], es el núcleo de Dilithium. La idea es construir una firma de la forma $\mathbf{z} = \mathbf{y} + c\mathbf{s}$, donde $\mathbf{y}$ es un vector aleatorio, c es un desafío hash y $\mathbf{s}$ es el secreto. El problema es que $\mathbf{z}$ podría filtrar información sobre $\mathbf{s}$ si sus coeficientes son demasiado grandes. Similar a un análisis de frecuencias en criptografía clásica, un atacante podría observar muchas firmas y deducir patrones.

La solución es **abortar** la firma y reintentar cuando $\mathbf{z}$ supera un umbral $\gamma_1 - \beta$, de forma que la distribución final de $\mathbf{z}$ es independiente del secreto:

Teorema 2.1 (Seguridad por rechazo). *Si $\mathbf{y} \xleftarrow{\$} [-\gamma_1, \gamma_1]^l$ y se rechaza $\mathbf{z}$ cuando $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$, entonces la distribución de $\mathbf{z}$ aceptado no depende de $\mathbf{s}$.*

Capítulo 3

El Esquema CRYSTALS-Dilithium

3.1. Descripción general

Dilithium implementa un esquema de firma digital con tres operaciones: generación de claves (**KeyGen**), firma (**Sign**) y verificación (**Verify**). La seguridad se reduce a la dureza de M-SIS (infalsificabilidad) y M-LWE (privacidad de la clave).

3.2. Parámetros

Cuadro 3.1: Variantes de Dilithium y sus parámetros principales.

Variante	(k, l)	Seguridad	$ pk $	$ \sigma $
Dilithium2	(4, 4)	128 bits	1.312 B	2.420 B
Dilithium3	(6, 5)	192 bits	1.952 B	3.293 B
Dilithium5	(8, 7)	256 bits	2.592 B	4.595 B

3.3. Generación de claves: KeyGen

El algoritmo de generación de claves parte de una semilla aleatoria $\xi \xleftarrow{\$} \{0, 1\}^{256}$ de la que se derivan, mediante una función hash, tres valores: ρ , ρ' y K . A partir de ρ se expande de forma determinista la matriz pública $\mathbf{A} \in R_q^{k \times l}$, y a partir de ρ' se muestrean dos vectores de polinomios pequeños $\mathbf{s}_1 \in S_\eta^l$ y $\mathbf{s}_2 \in S_\eta^k$, donde S_η denota el conjunto de polinomios con coeficientes acotados por η .

Con estos elementos se calcula $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$, que actúa como compromiso público. Para reducir el tamaño de la clave pública, $\mathbf{t}$ se descompone en una parte alta $\mathbf{t}_1$ y

una parte baja $\mathbf{t}_0$ mediante la función **Power2Round**. La clave pública es $pk = (\rho, \mathbf{t}_1)$ y la clave privada es $sk = (\rho, K, \mathbf{t}_0, \mathbf{s}_1, \mathbf{s}_2)$.

3.4. Firma: Sign

Para firmar un mensaje M , se calcula primero un hash $\mu = H(H(pk) \| M)$ que vincula el mensaje a la clave pública. A continuación se ejecuta un bucle de intentos basado en el mecanismo de Fiat-Shamir con abortos descrito en la sección 2.5.

En cada iteración se muestrea un vector aleatorio $\mathbf{y} \xleftarrow{\$} S_{\gamma_1-1}^l$ y se calcula $\mathbf{w} = \mathbf{A}\mathbf{y}$. De la parte alta de $\mathbf{w}$ se obtiene el desafío $c = H(\mu \| \text{HighBits}(\mathbf{w}, 2\gamma_2))$, y se construye la respuesta $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$.

Si $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$, la firma se descarta y se reintentan con un nuevo $\mathbf{y}$. Si la norma es aceptable, se calculan las pistas h necesarias para la verificación y se devuelve la firma $\sigma = (\mathbf{z}, h, c)$.

3.5. Verificación: Verify

Para verificar una firma $\sigma = (\mathbf{z}, h, c)$ sobre un mensaje M , el verificador comprueba en primer lugar que $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, rechazando directamente cualquier firma con coeficientes fuera del rango esperado.

A continuación reconstruye el compromiso original calculando $\mathbf{w}' = \mathbf{A}\mathbf{z} - c\mathbf{t}$, y obtiene su parte alta usando las pistas h . Con ella recalcula el desafío $c' = H(\mu \| \text{HighBits}(\mathbf{w}', 2\gamma_2))$ y acepta la firma si y solo si $c = c'$, es decir, si el desafío reconstruido coincide con el de la firma.

3.6. Repo Github

El siguiente enlace lleva al directorio de github con la implementación.

https://github.com/Javiifdez/Proyecto_Cripto_Dilithium.git

Capítulo 4

Conclusiones

En este trabajo se han presentado los fundamentos matemáticos de las retículas modulares y el problema M-SIS, y cómo estos permiten construir un esquema de firma digital resistente a ataques cuánticos. La técnica de *Fiat-Shamir con abortos* es la contribución más elegante de Dilithium: garantiza que la firma no filtra información sobre el secreto sin necesidad de primitivas adicionales.

Bibliografía

- [1] Shor, P. W. (1994). *Algorithms for Quantum Computation: Discrete Logarithms and Factoring*. Proceedings of the 35th Annual Symposium on FOCS, pp. 124–134.
- [2] Lyubashevsky, V. (2009). *Fiat-Shamir with Aborts: Applications to Lattice and Factoring-Based Signatures*. ASIACRYPT 2009, LNCS 5912, pp. 598–616.
- [3] Micciancio, D. y Regev, O. (2007). *Worst-Case to Average-Case Reductions Based on Gaussian Measures*. SIAM Journal on Computing, 37(1), pp. 267–302.
- [4] NIST (2016). *Post-Quantum Cryptography Standardization*. <https://csrc.nist.gov/Projects/post-quantum-cryptography>
- [5] Ducas, L. et al. (2022). *CRYSTALS-Dilithium: Algorithm Specifications and Supporting Documentation*. <https://pq-crystals.org/dilithium/>
- [6] NIST (2023). *FIPS 204: Module-Lattice-Based Digital Signature Standard (Draft)*. <https://doi.org/10.6028/NIST.FIPS.204.ipd>